GARDEN AUTOMATION

Garden Sprinkler Controller

Reference edition

Version 1.4.0 | Updated 14 November 2026

A dark-mode CustomTkinter desktop app for seven independently named sprinkler zones plus a channel 8 master pump, with an optional second eight-channel **Waveshare Modbus POE ETH Relay (B)** or individual **Tapo P110M Wi-Fi plugs** for independent channels 9-16. The headless controller runs as a Linux user service, including while logged out. The desktop app connects locally to that service; it never owns the relays. An optional phone web interface can be reached privately over Tailscale.

> **Before you begin**
> Closing the app does not stop watering. Use the relevant station/group stop or Stop Everything. Garden and appliances have separate hardware fault handling, but share the controller computer. The controller computer must stay awake. Software cannot guarantee that a valve or pump switches off during a power, network or equipment failure.

Includes compact phone controls, independent fault groups, explicit appliance retry, off-peak hot-water reconciliation and 24-hour Tapo graphs with 15-minute measurements.

Developed by Wally Horsman, 2026
Prepared from the current README documentation, updated 14 November 2026.
This guide covers the sprinkler application, not an Ubuntu system image.

Contents

1. Everyday controls at a glance 3
2. Setup and launch 3
3. Optional Tapo P110M plugs on channels 9-16 5
4. Tapo details and 24-hour graphs 7
5. Independent garden and appliance fault groups 8
6. Hot Water System on an intermittent off-peak supply 8
7. Optional phone control over Tailscale 9
8. Optional second relay board (channels 9-16) 12
9. Your Waveshare relay 13
10. Using the controller 14
11. Channel 8 master pump 20
12. Timing and operational limits 20
13. Code layout 21
14. Tests 22

> **Backup or timetable?**
> A timetable stores named schedules. An application backup is an encrypted recovery archive with settings, credentials and history. Keep its password safe.

Contents entries and web references are clickable. Command examples are intended for a terminal on the controller computer; read the surrounding instructions first.

1. Everyday controls at a glance

Control or status	What it means
Stations 1-8	Seven garden zones and the automatic station 8 master pump. Garden hardware faults stop only this group when 16 outputs are configured.
Stations 9-16	Independent appliance outputs. An ordinary plug fault affects only that station, not the garden or other working appliances.
Stop Garden (1-8)	Attempts OFF only for the garden group, pump first. Current garden schedule windows are suppressed.
Stop Appliances (9-16)	Attempts OFF only for appliances. Current appliance schedule windows are suppressed.
Stop Everything	Attempts OFF for both groups. An unreachable output cannot confirm OFF.
Retry station / RETRY STATION (OFF)	Confirms an ordinary faulted appliance OFF before clearing its fault. It does not restart a missed run.
Rain Pause	Stops/blocks included stations in either group. Saved exemptions remain active.
Show Tapo Details	Read-only measurements and clickable 24-hour graphs; failed telemetry is a gap, not zero.

The phone's three red stop buttons use smaller text in one compact, three-across row. They retain separate confirmations and tap targets at least 44 CSS pixels high. Garden and appliance health are shown separately below them. Closing the desktop, signing out or leaving the phone page does not stop equipment. Service shutdown/restart, restore hold and shared controller failures can still affect both groups; they share one computer and controller service.

2. Setup and launch

Requires Linux with systemd and Python **3.11 or newer**. Only the desktop app requires Tk, CustomTkinter, and a graphical session. The service uses the installed requirements and does not need a display. On Debian/Ubuntu, install the Python desktop/virtual-environment prerequisites:

```
sudo apt install python3-tk python3-venv
```

From this folder:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
mkdir -p ~/.config/systemd/user
cp garden-sprinkler.service ~/.config/systemd/user/
loginctl enable-linger "$USER"
systemctl --user daemon-reload
systemctl --user enable --now garden-sprinkler.service
.venv/bin/python app.py
```

Close any older, direct-hardware version of the app before starting the service. The supplied unit expects this project at `~/Documents/garden-sprinkler-controller`; edit its paths if installed elsewhere. Enabling linger may require administrator approval on other machines. Do not move/delete this project or its virtual environment while it is installed.

Useful service commands:

```
systemctl --user status garden-sprinkler.service
journalctl --user -u garden-sprinkler.service -n 50 --no-pager
systemctl --user restart garden-sprinkler.service
systemctl --user disable --now garden-sprinkler.service
```

Linger starts the user service manager at boot, without requiring a login. The enabled service survives logout and restarts after a process failure. Failed startup (for example the network/relay still booting) retries every 15 seconds. Runtime hardware faults latch the affected garden group or appliance station rather than repeatedly retrying ON commands; see the independent-group policy below for garden restart versus appliance retry. Opening, closing, or reconnecting the desktop does not reset outputs. To stop garden watering, use **Stop Garden (1-8)** or Rain Pause; use **Stop Everything** for both groups, not the window close button. After updating the project, restart the background service before reopening the desktop so both processes use the same settings format.

The PC must remain awake. This installation does not change machine-wide sleep or BIOS/UEFI settings. For recovery after a mains failure the PC must actually power up again: enable "Restore on AC Power Loss"/"AC Back" in firmware if needed. The service cannot turn on a powered-off computer.

To change relay/configuration parameters, use `systemctl --user edit garden-sprinkler.service` and override `ExecStart` (first clear it with an empty `ExecStart=`). The `--config`, `--host`, `--port`, `--slave-id`, and `--timeout` arguments now belong to **service.py**, not app.py. Restart the service afterward. For foreground diagnosis only, stop the installed service before running `.venv/bin/python service.py`; never run a second hardware controller. The desktop accepts `--socket` for an alternate local service endpoint.

The default is `~/.config/garden-sprinkler-controller/settings.json`. Settings are validated and written using atomic replacement. Missing settings start with eight disabled routines. Invalid existing settings produce an error and are never silently overwritten.

3. Optional Tapo P110M plugs on channels 9-16

This option keeps the existing Waveshare board on channels 1-8 and uses direct local Wi-Fi control for up to eight P110M plugs. No Home Assistant, Zigbee hub, or Matter controller is required. It uses the community `python-kasa` library, not a TP-Link-supported API. P110M AU hardware 1.0 / firmware 1.2.3 is listed in the library's tested devices; the actual arriving hardware/firmware still needs testing. Read-only energy monitoring and local measurement history are integrated; Matter pairing is not.

Newer TPAP firmware: The normal dependency does not support every P110M. P110M firmware 1.4.4 was authenticated and switched in isolated tests and the experimental environment is in use on this installation with three assigned plugs. It uses an unmerged experimental `python-kasa` fork, pinned in `requirements-tapo-experimental.txt`. This fork is not a stable upstream release. The standard `requirements.txt` remains unchanged; do not install both requirement files into the same environment. To prepare a separate experimental environment, run:

```
python3 -m venv .venv-tapo
.venv-tapo/bin/python -m pip install -r requirements-tapo-experimental.txt
```

Using it for the controller requires a systemd user-service `ExecStart` override pointing to `.venv-tapo/bin/python` and a restart. Stopping or restarting interrupts current watering; startup attempts OFF on every assigned output and active saved schedules can start again. Plug operations have a 5-second overall limit plus bounded connection cleanup. Operations are sequential and can delay other controller work; a timeout can leave the plug state unknown and latch that appliance station's fault. No Wi-Fi software solution guarantees OFF after network or controller failure. Test with no equipment attached before use.

Full-system backups record service overrides but currently include only the standard `requirements.txt`, not the experimental requirements file or virtual environment. For recovery, separately retain the experimental requirements file and recreate `.venv-tapo` before starting a restored service that uses it.

Prepare the slots before plugs arrive:

```
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python tapo_plugs.py prepare
systemctl --user restart garden-sprinkler.service garden-sprinkler-phone.service
```

Reopen the desktop or refresh the phone. New channels 9-13 are named **Tapo Plug 1** through **Tapo Plug 5**, with schedules disabled; 14-16 are spare. All eight remain explicitly **Unassigned** until connected. Names, schedules, Rain Pause exemptions, manual limits and reminders can be prepared now. Unassigned channels cannot run, including from schedules, and are not reported as successfully switched OFF. Existing channel names/settings are preserved. Tapo mappings and a second Waveshare board cannot use channels 9-16 together.

When the plugs arrive:

1. Pair each in the Tapo app on the controller's reachable local Wi-Fi network. Avoid guest/client isolation. Keep native Tapo/Matter schedules and competing automations disabled so they do not conflict with this controller.
2. Enter the TP-Link account credentials **locally**, never in chat:

   ```
   .venv/bin/python tapo_plugs.py credentials
   .venv/bin/python tapo_plugs.py discover
   ```

   The password prompt is hidden. Discovery reads device information only and does not switch plugs. Use `discover --target <LAN-broadcast-address>` if the controller has multiple network interfaces. If authentication fails, check any third-party compatibility/local-access setting in the Tapo app.
3. Reserve each plug's IPv4 address in the router. Discovery shows its IP, MAC and Tapo name. Assign one unique IP/MAC to each channel, starting at 9:

   ```
   systemctl --user stop garden-sprinkler.service
   .venv/bin/python tapo_plugs.py assign 9 192.168.0.50 12:34:56:78:9a:bc
   .venv/bin/python tapo_plugs.py show
   systemctl --user start garden-sprinkler.service
   ```

   Replace both example identifiers with the real plug's values; repeat `assign` for any additional channels 10-16 while stopped. Starting the controller attempts **OFF on every assigned output**; saved active schedules can then start. Make the first test with no equipment attached. MAC and P110M model checks precede switching, and state readback must match the request. Writes are not retried.
4. Test each real plug individually before connecting a load. Use only loads within its Australian electrical ratings, including motor/inrush limits. Channels 9-16 never activate the sprinkler master pump. Continuous runs on these independent channels do not need a pump-exclusion flag, but are blocked by a saved manual maximum and by Rain Pause unless exempt.

For alternate settings use `tapo_plugs.py --config /path/to/settings.json ...` with the same path as the service. Assignments are stored in the separate `settings.json.tapo.json`; account credentials are stored in `settings.json.tapo-credentials.json`, owned by the current user with no group/world access (normally mode `0600`). Both are gitignored. Full-system backups include them only inside the password-encrypted archive; ordinary status and phone responses never expose these credentials. Protect the archive and keep its password separately. `prepare` preserves existing assignments. Changing credentials requires a controller restart.

Stop the service with the old plug still reachable before reassigning or using `tapo_plugs.py unassign 9`. Removing an assignment cannot turn off a disconnected plug. Runtime does not rediscover changed IPs automatically; use DHCP reservations. After expansion, export a new 16-channel backup: old eight-channel backups do not match the expanded channel count. Existing eight-channel timetable profiles remain usable in the channels 1-8 group.

Failure and status limits: Stations 1-8 and 9-16 have independent hardware fault handling. A plug switching/status-read failure faults only its station; garden watering and other working appliances remain enabled. An unreachable plug at startup does not prevent garden operation. OFF cannot be guaranteed during Wi-Fi loss or loss of controller power; do not use this as a safety interlock. Commands are bounded, sequential network operations, not hard real-time switching. Normal status shows controller runs/last requested state, not continuous polling of changes made by a physical button or another app. Desktop **Read Relay Status** performs a fresh read. Verified plug state is not proof that the connected appliance operates or that electrical power is isolated.

4. Tapo details and 24-hour graphs

Show Tapo Details opens a read-only table of all assigned P110M plugs on the desktop and a separate view on the phone. It shows reported ON/OFF state, power in watts, today's and this month's energy in kWh, voltage, current and the reading time. Opening the view or pressing **Refresh** requests one batch of readings; overlapping requests are combined and batches are limited to one per ten seconds. Reads run outside the controller's switching loop. Missing measurements are shown as unavailable, not zero. Failed refreshes retain previous values marked stale and do not stop watering. This does not change the controller's existing fault policy for switching failures. Readings are not proof of electrical isolation or appliance operation.

Click a plug row (or select it and press Enter) for its last 24 hours of **controller switching history**. Desktop opens a graph window; phone opens a graph with Back to Tapo table. ON/OFF bands, local event times, period durations, reasons and total recorded ON time come from the retained activity log. Unrecorded periods, restarts and faults are Unknown until another switch event is recorded. The log retains at most 1,000 events, so a complete day is not guaranteed. Changes made through the physical button or Tapo app are not tracked; this is not a historical power-usage graph. Refresh history updates the view without reading or switching plugs.

The controller now also records a snapshot from each assigned plug every 15 minutes, even with desktop and phone closed. Sampling starts in the current quarter-hour slot after startup and runs in the telemetry worker, not the switching loop. The graph's measurement selector shows watts, volts, amps, reported state or daily/monthly energy counters, with a table of actual reading times. These are snapshots, not interval averages; brief runs can be missed. Missing or failed samples break the line, rather than becoming zero. Counters can reset at day/month boundaries and are not interval consumption. Recording cannot reconstruct earlier readings. The separate private `settings.json.tapo-samples.sqlite3` retains 25 hours, survives restarts and separates readings by plug MAC. It is not currently included in full-system backups. Sampling/storage failures are reported without latching a watering fault; switching failures follow the independent group/station policy below.

5. Independent garden and appliance fault groups

When 16 outputs are configured (Tapo or a second relay board):

- **Garden, stations 1-8:** a relay/pump command or status-read failure latches the garden group OFF and attempts to stop only this group, pump first. Appliance runs and hot-water requests are not cancelled. Check the garden hardware and restart the controller before resuming garden operation.
- **Appliances, stations 9-16:** a failed command or explicit status read latches only that station's fault, cancels its logical run and marks its history Unknown. A failed ON receives one safety OFF attempt, not a repeated ON. No automatic restart/retry is performed for ordinary plugs. Reconnection or a successful read alone does not clear the fault.
- **Retry station (confirm OFF)** explicitly attempts OFF. Only confirmed success clears an ordinary appliance's fault. The missed run is not resumed, and a currently active schedule window stays suppressed; use Run for a new manual request or wait for the next window. A service restart also attempts OFF and can resume active saved schedules for successfully initialized stations.
- Zone 16's explicitly enabled off-peak policy retains automatic reconciliation of the current request, as described below, instead of latching ordinary appliance faults.

Desktop and phone show garden health and individual appliance faults, without disabling unaffected controls. **Stop Garden (1-8)** stops/suppresses only garden outputs; **Stop Appliances (9-16)** stops/suppresses only appliance outputs. **Stop Everything** (the former STOP ALL) attempts OFF on both groups, even if an output is unreachable. Failed OFF is never proof of physical isolation. Rain Pause still applies across groups according to saved exemptions.

This is fault isolation, not two independent computers/processes: the controller, configuration, storage and power are shared. Service restart/shutdown, restore safety hold and shared controller/storage failures can still stop both groups. Bounded synchronous plug operations can briefly delay the shared loop; this is not hard real-time separation. An eight-output-only installation keeps its original whole-garden fail-safe behaviour.

6. Hot Water System on an intermittent off-peak supply

Zone 16 can explicitly opt into a device-bound off-peak policy. Only use this after an electrician has confirmed the plug, connection and hot-water load are suitable and automatic reconnection is acceptable. With the controller stopped, assign its verified IP/MAC using `.venv-tapo/bin/python tapo_plugs.py assign 16 IP MAC --off-peak` (or the standard environment for firmware it supports). The optional `off_peak: true` field is retained in encrypted full-system backups. Reassigning without this option or unassigning removes the policy; it cannot be enabled on any other zone and is not inferred from the zone name.

For this output only, switching requests are queued separately from confirmed plug state. An unreachable plug does **not** latch a sprinkler fault or stop the garden pump. Other appliances use individual latched faults; garden hardware faults remain within stations 1-8. Desktop/phone status and Tapo Details show requested ON/OFF, connection availability and the last confirmed state/time. Unknown is never presented as confirmed OFF. Reachable and ON do not prove the heater is heating. For isolated zone 16, **Read Relay Status** returns the background worker's latest confirmation, not a synchronous new network read; check its confirmation timestamp in zone status or Tapo Details.

A single background worker checks at startup, on request changes and about every 30 seconds. When reachable, it applies **only the current request**, never a queue of missed commands. Timer/schedule expiry, Stop, Stop Appliances, Stop Everything, a shared controller fault, restore hold and shutdown request OFF; Rain Pause still respects the exemption. A garden-only fault or Stop Garden does not cancel the hot-water request. A command already in flight cannot be recalled, but a superseding OFF cancels an unsent ON and is reconciled after an in-flight operation. Shutdown waits a bounded time for OFF and logs explicitly if it cannot be confirmed.

The graph separates requested operation from last-confirmed ON/OFF history. An unreachable observation starts an Unknown period; no definite loss-of-supply time or cause is inferred. Outages can occur between checks. Last confirmation is a historical observation, not continuous proof. The 15-minute measurement graphs remain snapshots with gaps for failed readings.

An unpowered plug cannot receive commands. "Unavailable" could mean off-peak supply loss, Wi-Fi trouble, credentials or firmware, not proof of any one cause. This feature does not inspect or change the plug's independent power-restoration setting, and cannot guarantee it stays OFF when supply first returns. Confirm that behaviour separately before relying on unattended hot-water control.

7. Optional phone control over Tailscale

The optional phone interface lets you check channel status, start a manual timed run, switch a channel ON until stopped, stop an active channel, or stop all watering from a mobile browser. It does not replace the background controller. The web server talks to that controller over its existing private local socket and binds only to `127.0.0.1`; it cannot be opened directly to the LAN or internet.

The home page shows all 16 channel positions in a compact three-column grid (two columns on screens narrower than 360 CSS pixels). Each button shows a channel number, shortened name and text status. Unconfigured positions remain visible as unavailable. Tap a button to open a dedicated view containing only that channel's full name, status and controls, plus **Back to all zones** and the compact **Stop Everything / Stop Garden / Stop Appliances** row. The phone/browser Back action returns directly to the home grid; zone visits do not build a trail of screens. At home, press Back twice within two seconds to leave; the first press displays a reminder that watering continues. Waiting longer resets the exit confirmation. Leaving returns to the previous browser page where available; it does not forcibly close a browser tab. Selecting a channel never starts watering. Only one control panel is open at a time, and duration/reminder drafts are retained when switching channels. The duration picker stays open during automatic status refreshes until you choose a time

or dismiss it. Changes affecting that zone's controls (such as a fault, Rain Pause or a run started elsewhere) still update them immediately. After choosing a duration, tap **RUN** once to start; leaving the picker does not rebuild the controls or interrupt the tap. When RUN is unavailable, the zone page explains why. In particular, Rain Pause blocks manual watering for non-exempt zones: return to **Back to all zones** and turn Rain Pause off if watering is intended. Schedule-group pauses are separate and do not override Rain Pause. Channel 8 shows automatic pump status rather than independent run controls. The zone view has four **Not Paused / Paused** schedule-group buttons matching the desktop. Each applies to its numbered Start/Stop group across all weekdays, not other groups, other zones or manual runs. Paused buttons are red. Unpausing waits for the next scheduled start rather than resuming the current period. Changes save immediately and reject stale edits from another window. Reminder settings expand only when needed. The three stop buttons sit **three across**, with smaller text and at least 44 CSS pixels of tap height, on both home and station pages. **Rain Pause** and the running summary stay above the home grid. The group-health line distinguishes garden faults from individual appliance faults. A faulted ordinary appliance has **Retry station (confirm OFF)**; its RUN controls are disabled until that explicit retry succeeds. Working stations remain usable. A garden fault requires checking the hardware and restarting the controller; appliance controls remain available. Rain Pause changes require confirmation and preserve saved zone exemptions. Activity history opens in its own view only when its home-page button is tapped. Events are fetched only while requested, not automatically at sign-in. Filters, Older/Newer and Refresh remain available; Back returns directly to the home grid. Recording continues in the controller regardless of whether the log is open. Backup is in an expandable section; backup opens automatically during a restore hold so the resume control remains discoverable.

If the page ever appears plain white with basic text and no normal colours or layout, reload it to request the stylesheet again. If it persists, check Tailscale connectivity and the phone service before changing settings. Refreshing the page does not stop equipment. After a phone-service restart, sign in again.

Install and sign in to Tailscale on the controller PC and Android phone using your tailnet account. Tailscale Serve requires HTTPS certificates to be enabled for the tailnet; its first-use flow can prompt you to enable them. Keep the tailnet access policy restricted to people and devices you trust.

From the project folder, create the additional web password and start the optional user service:

```
mkdir -p ~/.config/garden-sprinkler-controller ~/.config/systemd/user
chmod 700 ~/.config/garden-sprinkler-controller
python3 -c 'import secrets; print(secrets.token_urlsafe(36))' > ~/.config/garden-
sprinkler-controller/phone-password
chmod 600 ~/.config/garden-sprinkler-controller/phone-password
cat ~/.config/garden-sprinkler-controller/phone-password
cp garden-sprinkler-phone.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now garden-sprinkler-phone.service
```

The password file must be owned by your account, have mode `0600`, and contain at least 4 characters. A long generated password is recommended; a short PIN is easier to guess and should only be used with trusted Tailscale devices. After five incorrect attempts within five minutes, new

sign-ins are blocked until the failed attempts age out of that window. This limit is shared across all phones; already signed-in sessions continue to work. The `cat` command prints the generated password so you can save it in a password manager and enter it on the phone; keep it private. The web interface also requires this password even when accessed from the tailnet.

To change it, edit `~/.config/garden-sprinkler-controller/phone-password`, keep its permissions at `0600`, and run `systemctl --user restart garden-sprinkler-phone.service`. This signs out existing phone sessions without restarting the sprinkler controller.

Publish the loopback-only web server to your tailnet with Tailscale Serve:

```
tailscale serve --bg 8765
tailscale serve status
```

Use the HTTPS `https://<computer>.<tailnet>.ts.net` URL shown by the status command in Chrome on the Android phone, with Tailscale connected. The browser will ask for the phone-control password. Tailscale Serve is tailnet-only; do not use Tailscale Funnel or configure router port forwarding for this service.

Choose **10 minutes**, **30 minutes**, **1 hour**, or **Custom**, then press **RUN**. For Custom on phone or desktop, enter minutes (for example `30` or `90`) or append `h` for hours (`2h`, `2.5h`, `0.5h`). Uppercase `H` and surrounding spaces are accepted. Without `h`, the number means minutes. The summary shows the interpretation: `2.5h` is **2 hours 30 minutes (150 minutes)**. The result must be 1–1440 whole minutes (up to `24h`); fractional-minute results are rejected, never rounded. Saved per-channel maximums still apply. The default is 10 minutes. The computer can set an optional per-channel maximum; requests above it are rejected. **RUN UNTIL STOPPED** requires confirmation and has no automatic time limit. While active, its button turns blue and reads **Now running until stopped**; use the separate **STOP** button to end the run. For channels 1-7 it is offered only when the channel's master pump exclusion has been saved on the home computer. Independent channels 9-16 do not need that exclusion. It is disabled when a manual maximum is set. The controller rejects continuous starts that violate these requirements. Removing a primary channel's exclusion stops its active continuous run instead of turning on the pump. Rain Pause exemption is a separate setting and is not required unless Rain Pause is enabled. It continues if you close the page, sign out, or disconnect your phone. Use **STOP**, the relevant group stop, or **Stop Everything** to end it. Per-channel master pump exclusions and Rain Pause exemptions apply to both options: pump-excluded channels do not start the pump, and Rain Pause stops/blocks non-exempt channels. Continuous runs are not saved and do not resume after controller restart; hardware faults stop the affected garden group or appliance station, not unrelated outputs. Saved schedules may independently start after a restart. If remote access is lost, use the computer's desktop controls to stop the channel.

Restart both the controller and phone services and reopen the desktop app when installing this change. Restarting the controller attempts to turn all outputs off; active saved schedules may start again, but manual runs do not resume. Choose a safe time to restart. Manual timers are disabled until the updated controller is running. Settings saved by the updated app use version 9 and cannot be read by an older controller. Service protocol 2 represents continuous remaining time as JSON `null`, and older desktop processes will report an incompatible service until reopened. The phone's continuous button is disabled until the updated controller runs. Channel 8's master pump remains automatic and cannot be manually switched independently. The phone interface is unavailable when

the PC or its Tailscale connection is offline, but this does not stop the sprinkler controller or its schedules. To disable phone access, run `tailscale serve reset` and then `systemctl --user disable --now garden-sprinkler-phone.service`.

8. Optional second relay board (channels 9-16)

The desktop's **Channels 1-8 / Channels 9-16** selector switches between banks. Without a second board configured, channels 9-16 are labelled **Board 2 not configured** and cannot be run or edited. No second-board network requests are sent, and the original board continues operating normally.

To enable them, give a second compatible eight-channel board its own IP address, check it read-only with `.venv/bin/python waveshare_relay.py --host 192.168.0.100` (replace this example IP with the actual address), then run:

```
systemctl --user edit garden-sprinkler.service
```

Add an override, retaining any existing primary-board/configuration arguments:

```
[Service]
ExecStart=
ExecStart=%h/Documents/garden-sprinkler-controller/.venv/bin/python
%h/Documents/garden-sprinkler-controller/service.py --second-host 192.168.0.100
TimeoutStopSec=60
```

Optional arguments are `--second-port` (4196), `--second-slave-id` (1), and `--second-timeout` (1 second). Defaults are independent of the primary board's options. Then restart the service and reopen the desktop:

```
systemctl --user daemon-reload
systemctl --user restart garden-sprinkler.service
systemctl --user status garden-sprinkler.service --no-pager
```

App channels 9-16 map to physical relays 1-8 on the second board. They support names, schedules, autosave, manual operation, and Rain Pause exemptions just like other zones, but **never operate channel 8's master pump**. Channel 8 serves only zones 1-7. Daily and weekly overviews include configured second-board channels. New second-board schedules start disabled. Settings for either board count are saved in version 9, including per-zone master pump exclusions, manual limits, and optional continuous-run reminders.

To disable the second board, stop the service while it is still reachable (so its outputs can be switched OFF), remove its `--second-*` arguments, and restart. Stored names and schedules for channels 9-16 are preserved but ignored until configured again. Restoring the second board can immediately resume its active saved schedules. Removing configuration does not switch off an unreachable or disconnected board.

A configured board failure is **not** treated as an absent optional board: primary-board faults stop the garden group; secondary-board command failures fault the affected appliance station, and a failed secondary-board status read faults stations 9-16. Neither group's hardware faults stop the other group. Check the hardware and logs before restarting. Device ownership locks cover both endpoints, preventing another service instance from owning either board.

9. Your Waveshare relay

The supplied device-settings screenshot confirms:

Setting	Value
IP address	`192.168.0.99`
TCP port	`4196`
Work mode	TCP Server
Transfer Protocol	None (Modbus RTU-over-TCP, **not** Modbus TCP framing)
Internal serial configuration	115200, 8 data bits, no parity, 1 stop bit

Leave the board settings unchanged. Zones 1-7 map to relays 1-7 (coil addresses 0-6). Relay 8 (coil address 7) is reserved for the master pump.

The manufacturer's protocol documentation shows slave address **1** in its examples. A read-only status request to your relay at `192.168.0.99:4196` confirmed that slave ID **1** responds and all eight relays reported **OFF**. No relay write commands were sent during this check.

First run a **read-only** status check from a computer on the same LAN. This command needs only Python, not Tk or any third-party packages:

```
python3 waveshare_relay.py
```

If address 1 does not respond, optionally discover a responding slave:

```
python3 waveshare_relay.py --discover
```

Discovery only sends function 01 (read eight coil states), tries addresses 1–247, and stops at the first valid response. It does not send writes or change settings. It can take several minutes if the device is not responding. Cancel with Ctrl+C. If no address responds, check network access, relay power, and port/protocol settings; a failed scan does not establish that the slave ID is wrong.

Open the service-connected desktop:

```
.venv/bin/python app.py
```

The service connects to `192.168.0.99`, port `4196`, slave ID `1`. Service startup attempts OFF on configured outputs and verifies normal synchronous commands. When 16 outputs are configured, an unreachable appliance faults only its station; a garden hardware failure latches the garden group, not working appliances. Off-peak hot-water OFF is requested and confirmed asynchronously. Saved routines that are scheduled now can start on healthy outputs, except zones included in Rain Pause or blocked by a restore hold. Exempt zones can therefore energize a valve immediately on service start. The **Read Relay Status** button queries and displays the device-reported logical states through the service without intentionally changing them. A failed explicit read triggers the affected group/station fault policy, which can send safety OFF commands. This is different from the non-fault-latching telemetry reads in Show Tapo Details. Isolated hot-water state comes from its background worker's last confirmation, with a separate confirmation timestamp.

There is no simulation mode or direct-hardware fallback. If the service is unavailable, the app displays an explicit error. If connection is lost while open, it marks status unknown, disables watering controls and retries its local connection every second. The service may still be watering while disconnected. Request timeouts are not retried: a command may have succeeded before its reply was lost. Status polling reconciles the result. The local Unix socket is private to the user (mode 0600); there is no network control listener. File locks prevent duplicate service owners using the same configuration or relay host/port. Multiple desktop windows are supported; saving stale edits is rejected rather than overwriting another window's changes.

The network adapter in waveshare_relay.py uses RTU CRC16, checks response addresses, functions, CRCs, write echoes, and coil readback. Requests use a fresh TCP connection and a bounded transaction timeout (default 1 second, configurable with `--timeout`, maximum 10). ON commands are never retried or treated as successful after a timeout; fault handling attempts OFF within the affected garden group or appliance station instead. Coil readback confirms the device's reported logical state, **not** valve movement, water flow, or contact integrity. Other programs and the board's digital inputs can change outputs; use command-compatible, non-linkage relay modes and avoid competing controllers.

Hardware communication errors follow the garden/appliance fault policy above. An eight-output controller stops the whole garden; with 16 outputs the groups are isolated. The app shows affected group/station faults and uncertain output states. Stop Everything remains available to attempt OFF commands. Requests are synchronous: the GUI may pause during network operations; a failed all-channel OFF sequence can take up to sixteen transaction timeouts with two boards, plus connection overhead. A successful write also requires a separate readback transaction.

10. Using the controller

- The selected channel status shows whether the output is commanded ON or OFF, its run source, an approximate end time for timed runs, and the next saved, enabled, unpaused scheduled start. The next start is a schedule prediction, not a guarantee: Rain Pause, STOP suppression, faults,

edits or another run may prevent it. The phone shows the same information for each channel; schedule times use the controller's local time. If status cannot be obtained, the interfaces show the output as unknown, not OFF.

- **Activity** on the desktop and **Activity history** on the phone show recent controller-recorded starts, stops and reasons, including manual timers, schedules, Rain Pause, group stops, Stop Everything, startup, shutdown and faults. Browse older pages or refresh to see new events. Newest events appear first, with dates and times shown as DD/MM HH:MM in the controller's local time at the event. Channel events show their channel number and name; system-wide events are labelled **System**. The most recent 1,000 events persist in `settings.json.activity.sqlite3` beside the settings file. History begins only after this version starts; it cannot reconstruct earlier activity. Readback reflects the relay's reported logical state, not actual lamp, fan or valve operation. Events may be missing after a crash, device disconnect, or storage failure; a history warning is shown and logged if recording fails. A history error does not stop relay control. Standalone direct-hardware desktop mode does not record history; the normal background service is required.
- **Daily Schedule**, above the editor, opens a weekday table of the selected channel bank's configured garden periods sorted by start time. It includes live unsaved edits (clearly labelled), excludes the master pump, and can hide disabled routines. Select a row to open its relay settings. Amber warnings flag identical starts and overlapping periods across enabled relays; adjacent end/start times are not overlaps. These are advisory only and do not block saving: when two enabled, unpaused schedules overlap, both relays can run at the same time. Watering is not queued sequentially; the shared master pump runs for their combined demand. Incomplete/invalid pairs remain visible with an explicit warning and are not fully conflict-checked. Rain Pause and stale service status are labelled; the overview shows configured times, not current relay outputs.
- **Weekly Overview**, beside Daily Schedule, shows the selected bank's garden relays against all seven days at once. Each day cell lists all four numbered start-stop periods (unused periods show `--`). Bright red text and `!` flag same-start or overlapping periods on both affected relays; red cells flag invalid or incomplete pairs. Click a garden or cell to open its settings. Both windows can remain open together and update as you edit. Both support hiding disabled routines, label unsaved edits, and exclude the master pump. Comparisons are within the same weekday, not across days. Both overviews follow the **Channels 1-8 / Channels 9-16** selector, including while already open, and identify the displayed bank in their summary. Channel numbers and navigation retain their original identities (9-16 are not renumbered). Conflict warnings still compare all configured enabled garden relays, including the other bank.
- The compact opening window is 940 x 760, with a minimum size of 940 x 620. One **RUN ZONE / STOP THIS ZONE** button handles timed watering and stopping. Stop Everything, separate garden/appliance stops and Save Settings remain in the footer. At smaller heights, scroll the configuration panel to reach lower fields using the mouse wheel over the panel (including on Linux) or the side scrollbar; the watering controls and Rain Pause remain visible outside that panel.
- Select each garden zone (1-7) on the left, edit its **Relay / Garden Name** at the top of the settings panel (up to 40 characters); valid changes save automatically about one second after you stop editing. The custom name appears beside that relay in the left-hand list and is saved separately

for each relay. The selected channel's panel also explains its master pump and Rain Pause settings and provides switches below the schedule. The summary marks changed exclusions as unsaved until applied. Channels 9-16 show pump independence rather than a pump switch; the sidebar controls remain available.

- Enable its routine, then set up to **four independent watering windows for each weekday** in the daily schedule. Matching start/stop pairs use muted blue, green, amber, and purple backgrounds to distinguish the four windows. Enter each start and stop in **24-hour time**, using **HMM** or **HHMM** (for example **815**, **0815**, or **0600**); **HH:MM** such as **08:15** is also accepted. Times display in **HH:MM** format. Leave both fields blank to skip a window; if you enter one time, its matching start or stop is required. Window numbers are independent: enter them in any order, but do not overlap windows within one zone. Windows cannot cross midnight. Each **Start** heading has its own **Not Paused / Paused** button directly below it. **Paused** has a red background and pauses that numbered Start/Stop group for this zone across all weekdays, preserving its times. Other groups, other zones and manual runs are unaffected. Once applied, pausing stops a scheduled run in that group; unpausing skips the currently active period and waits for its next scheduled start. Valid changes save automatically about one second after you stop editing; Save Settings applies them immediately. Group pauses are saved with the schedules, including in backups and timetables, and survive restarts. There is no whole-zone pause button below RUN ZONE or on phone channel cards. The former `settings.json.zone-pauses.json` sidecar is no longer used. Restart the updated controller service and reopen the desktop after this update. Red borders identify the time fields for overlapping windows. Clicking a conflicting Stop field opens an acknowledgment popup with the conflicting groups. Enabled routines need at least one complete time window; a different weekday can have different times.
- To reuse a day's schedule, choose that weekday in the menu above the schedule and check **Copy selected day's times to all days**. This copies all four windows once; each day remains independently editable afterward.
- The zone editor opens in **one day view**, showing today's single row with all four Start/Stop pairs. Use the weekday dropdown to display another day. **See week view** displays all seven rows; **See one day view** returns to the selected day. Your day and view selection stay the same when switching zones during the current app session. Switching days or views never changes times, copies settings, or triggers a save; hidden days are still validated and saved. Switching channel banks also switches the editor to the first channel in the newly selected configured bank, so its schedule and run controls match the bank.
- Older version 1, 2, 3 and 4 settings are migrated in memory, preserving existing weekdays and times in their original windows and leaving new windows blank. Existing zones remain included in Rain Pause. Version 5 and 6 settings also load without losing either board's configuration. Master pump participation is unchanged by migration; version 7 settings also load with no manual limit. Version 8 loads with reminders disabled; version 9 is written on the next save.
- Validation updates while typing. Valid edits are saved and applied automatically about one second after you stop editing, across **all stored zones and the preserved channel 8 settings**. Incomplete or invalid settings are not saved; fix the validation issue to resume autosaving. The **Save Settings** button remains available to save immediately. Closing prompts to save or discard any remaining unsaved changes.

- **Timetables** saves and loads named schedule profiles, such as **Summer** and **Winter**. Choose **Save current as...**, enter a name (1-40 characters), and confirm before replacing an existing profile. The window follows the **Channels 1-8 / Channels 9-16** selector and clearly labels its group. Saving/loading affects only that group's eight channels; the other group's saved schedules, selected profile and recovery timetable remain unchanged. Both groups can use the same profile name independently. Valid pending edits in the selected group are saved first; invalid edits in that group must be fixed. Pending edits in the other bank remain untouched (ordinary autosave still applies). Profiles contain the group's weekday times, routine enabled state, and paused-window flags. They do **not** change zone names, relay/board configuration, Rain Pause or pump exclusions, or global Rain Pause. Select a profile and choose **Load selected timetable**. After confirmation, settings are saved and applied immediately: scheduled relays may start or stop in that group, but existing manual/test runs retain their timers. The group's current timetable is saved automatically as **Before last load**; loading that recovery profile swaps it with that group's current timetable, allowing independent undo. The Timetable button, footer, and dialog show the selected profile and **modified** when its timetable has changed. Long names are shortened on the button; the dialog shows the full name. Normal autosave does not overwrite named profiles; save under the same name explicitly to update one. Preserved channel 8 schedules and unconfigured second-board schedules are included but remain inactive. Switching seasons is manual, not automatic. Profiles are stored beside the settings file as `settings.json.timetables.json`; include it with the settings and Rain Pause files when backing up or moving computers. A missing file means no profiles; a corrupt file prevents startup rather than silently discarding profiles. Older version-1 files are read without changing their schedules: eight-channel profiles go into channels 1-8; sixteen-channel profiles split into both groups retaining their names. The next profile save/load writes format version 2. Older app/service versions cannot read this format; update both together. Full-system backups cover both banks and include this named-profile file.
- Red indicates a commanded/confirmed energized relay, bright green an enabled schedule that is active at the current time, and grey no current watering window. A faulted garden group or ordinary appliance has a pale-red status dot. Isolated hot water has an amber dot while unknown or awaiting confirmation, red for confirmed ON and grey for confirmed OFF. Always read the status text; colours alone are not proof of electrical state. Sidebar schedule status uses the **saved/applied** routine.
- **RUN ZONE** runs the selected zone for the chosen duration, regardless of whether a routine is enabled. During Rain Pause it can run only for zones marked with **R-** in the left sidebar. Its button becomes **STOP THIS ZONE** whenever that zone is watering. Choose 10 minutes (default), 30 minutes, one hour, a custom duration from 1 to 1440 minutes, or **Until stopped**. Continuous operation requires confirmation, and channels 1-7 also require a saved pump exclusion. Independent channels 9-16 do not. The duration selector is to the left of the run button; choosing a duration does not start a run. For **Custom**, enter minutes or hours (for example **30** or **2.5h**) in the box that appears, then press **RUN ZONE**. A summary shows the selected duration before starting.
- **Running** on the desktop opens a live list of active channels, their source and approximate end time. The phone displays the list above the channels. The list reports controller state, not proof of physical device movement.

- **Backup** now creates an **encrypted full sprinkler-system backup**, on desktop and phone, rather than a settings-only JSON export. It includes both banks' saved settings and group pauses, all named timetables and selections, Rain Pause, relay connection details, Tapo assignments and credentials, the standard `phone-password` file beside settings, activity history, installed user service units/drop-ins, requirements and application source fingerprints. Unsaved desktop edits and active/manual runs are not included. Save edits first. This is application recovery, not an Ubuntu image: reinstall this repository and its requirements on a replacement computer; OS packages, the app executable, router/network settings, TLS/Tailscale configuration and unrelated OS secrets are not included. The separate 15-minute telemetry database is **not included**. The device-bound hot-water `off_peak` mapping flag is included. The experimental requirements file and virtual environment must be retained or recreated separately when the controller uses `.venv-tapo`.
  - Choose a **separate backup password of 12–1024 characters**. Several unrelated words work well. Enter it twice; **Show password** lets you check typing. It is not your Tapo or phone sign-in password. Store it in a password manager or a safe place separate from the backup. **Lost backup passwords cannot be recovered.** Neither password nor unencrypted backup contents are logged.
  - Keep the `.sprinkler-backup` file on another device. Files use authenticated AES-256-GCM encryption with a random salt/nonce and scrypt key derivation (N=32768, r=8, p=1). Exported and recovery files are user-only, mode `0600`.
  - Restore asks for the original backup password, validates all sections and previews what will be replaced before asking for confirmation. A wrong password, damaged archive or stale settings revision changes nothing. Old unencrypted settings-only JSON exports are not full-system backups and are rejected with an explicit message; retain them for manual settings recovery.
  - Before applying, an encrypted **before-restore recovery backup** is saved beside settings. It uses the **same password as the archive being restored**. All outputs are stopped before changes and a separate, persistent **restore safety hold** blocks schedules and manual/test/continuous runs, even Rain Pause-exempt plugs. A partial disk/write failure leaves the hold active and reports a fault; use the recovery backup to recover saved configuration.
  - Restart both services and reopen the desktop after restoration: `systemctl --user restart garden-sprinkler.service garden-sprinkler-phone.service`. Restarting never clears the hold. Inspect device assignments, schedules and Rain Pause, then explicitly select **Backup > Resume operation after restore** (or **Resume operation** on the phone). Resuming may start a current schedule immediately. No previous manual runs/countdowns are recreated.
  - Restored relay settings are read from the `.connections.json` sidecar and take precedence over command-line relay connection flags. Service units are saved as **reference copies**, not installed or executed automatically: usernames, installation paths, socket paths, custom environment variables and ports need review on another computer. The `.restored-service-config/RECOVERY.txt` directory alongside settings contains the setup guide, unit/drop-in copies and version fingerprints. Phone credentials at the standard location are restored; after the phone service restarts, sign in with the password saved in that backup. Custom `--password-file` locations outside the settings directory are not collected: copy the restored

standard password file there or adjust the service flag before restarting. Custom externally referenced secret files and network configuration need separate recovery. Backup never exports phone session cookies.

  - If the controller cannot start (or you are rebuilding on another computer), use the offline recovery tool after installing the app dependencies: `systemctl --user stop garden-sprinkler.service`, then `.venv/bin/python system_backup.py restore /path/to/file.sprinkler-backup --config /path/to/settings.json`. It prompts for the password without echoing it and requires typing `RESTORE`. It refuses to modify a configuration owned by a running controller and keeps the safety hold active. Offline disaster recovery does not create another before-restore archive; keep your original and the automatic recovery file. Use `inspect` instead of `restore` to check a backup without changing settings.

- Activity history can be filtered by channel, system events or faults; older years appear with the date. A lost connection means output states are unknown.
- Each channel can have an optional continuous-run reminder (1-1440 minutes), set below its desktop schedule or on its phone card. When a continuous run reaches that duration, an on-screen notice appears while the interface is open. The notice does **not** stop equipment or notify a closed app/page. Manual, test and scheduled runs do not trigger this reminder.
- Below the schedule, **Maximum manual run** optionally sets a per-channel limit of 1-1440 minutes; blank means no per-channel maximum. It applies to timed manual and test runs, not schedules. Requests exceeding it are rejected and Until stopped is unavailable. Adding or lowering a saved limit stops an active manual/test/continuous run; scheduled runs are unaffected. The limit must be saved before starting a manual run. Timetable profiles do not change it.
- **RAIN PAUSE** in the left sidebar stops scheduled and manual watering for included zones. Click **R** beside a zone number to exempt that zone; **R-** marks an exempt zone and turns red while Rain Pause is on. Exempt zones keep their schedules and can be watered manually during Rain Pause. Click **R-** again to include a zone. Zone exemptions save automatically with other valid settings. Rain Pause itself saves immediately; pending valid edits are saved first, and invalid edits must be fixed before enabling Rain Pause. Pause state survives restart. Turning it off asks for confirmation: a currently active saved routine may start immediately for its remaining time. Previous Stop This Zone/group-stop/Stop Everything suppression still applies; an interrupted manual run does not restart. Pause state is stored beside the selected settings file, with `.rain-pause.json` appended to its name (normally `settings.json.rain-pause.json`); include this file in backups. A missing pause file means not paused. An invalid file prevents startup; a settings-save failure can stop both groups, while a relay failure follows the garden/appliance fault policy.
- **STOP THIS ZONE**, the two **group stop** controls and **Stop Everything** send OFF requests immediately and suppress the affected stations' currently active automatic watering window(s) until they end. Group stops leave the other group's runs and schedule windows untouched. Network timeouts can delay OFF; an unreachable output is never guaranteed OFF. Later scheduled windows that day are unaffected. Another manual run can still be started, unless Rain Pause is on for that zone. Suppression is not persisted: restarting the service or PC during a saved window starts that routine again unless Rain Pause is enabled for that zone. Closing or reopening the desktop does not clear suppression.

- A manual watering run replaces that zone's current scheduled run; when the timed override ends, the routine does not resume in the same window. Disabling a routine and saving stops its scheduled run, but does not cancel an independent manual run; use **STOP THIS ZONE** for that.

11. Channel 8 master pump

Channel 8 is labelled **Master Pump**. It automatically switches ON whenever any non-excluded garden zone (1-7) is running, whether scheduled or manually. It has no independent schedule or watering control. Its red sidebar dot indicates the commanded/confirmed pump ON state.

Beside each zone's Rain Pause control is a separate **P** pump control. For channels 1-7, click it to switch between **P** (uses the pump) and **P-** (excluded from the pump). A saved exclusion is highlighted red. This setting autosaves independently of the Rain Pause exclusion: an excluded zone still runs its schedule and manual watering, but does not start or keep the pump running. If another non-excluded zone is running, the pump stays on. Changing only the pump exclusion during a run updates pump demand without restarting or cancelling the zone. Channel 8 and channels 9-16 have no pump exclusion control; channels 9-16 are already independent of the pump.

The app opens the first zone valve before starting the pump. The pump stays ON while non-excluded zones on channels 1-7 are running and stops before their last valve closes. Startup, Stop Garden, Stop Everything, garden fault shutdown and normal service shutdown attempt pump OFF before the garden valves. Stop Appliances and appliance faults do not change pump demand. Rain Pause keeps the pump on only when a Rain Pause-exempt, pump-using zone on channels 1-7 is running. There are no added valve-travel or pump delays; command order is not proof that a physical valve has finished moving. Failed garden ON commands stop the garden group and attempt its outputs OFF; appliances are not stopped. Shared controller failures can still stop both groups. Communication failures can prevent OFF commands from reaching the device.

The original channel 8 name and schedule are preserved in settings but ignored in pump mode. To restore eight independent garden zones, stop the service and set `MASTER_PUMP_CHANNEL = None` near the top of controller.py, then restart the service. The nearby comment explains this switch; the original scheduling and editing logic remains available, rather than being deleted. Restoring eight-zone mode also restores any enabled saved channel 8 routine, which may start immediately. Review it before restoring that mode.

Use a suitably rated pump contactor where required; do not assume the relay contacts can handle a pump's motor starting current. Have mains wiring and protection checked by a qualified electrician. Software does not replace independent dry-run, pressure, overload, or emergency-stop protection.

12. Timing and operational limits

The service must be running and the computer awake for schedules to run. It checks outputs every 250 ms when idle; network operations can delay these checks. Start/stop actions do not have real-time precision. It uses the operating system's **local time and weekday**. Starting the service during a scheduled window runs only its remaining time; missed windows are not replayed.

Routine runs end at their stop time or their originally calculated elapsed-time deadline, whichever occurs first. Manual/test durations use a monotonic clock. This prevents a backwards wall-clock adjustment from extending an active run. Clock changes and daylight-saving transitions can shorten or skip watering; this is not a timezone-aware scheduling service. Multiple zones may run simultaneously; the master pump follows the combined demand of channels 1-7 only. There is no sequential queuing or flow/pressure monitoring.

Normal service shutdown attempts OFF on all configured outputs; isolated hot water uses its bounded shutdown reconciliation. Closing the desktop leaves watering running. A killed service process, network failure, sleeping computer, or frozen event loop cannot guarantee shutdown. **Timers are software-only; they do not program a hardware auto-off or watchdog.** Use normally-open contacts and an independently designed hardware cutoff. Test with valve power disconnected first; verify correct channel mapping, command mode, readback, and shutdown before connecting actual valves. Never rely on the app alone to prevent flooding or protect a pump.

13. Code layout

The UI and scheduling code use the `RelayBackend` protocol in controller.py.

- app.py: dark-mode desktop service interface.
- service.py: sole hardware owner, headless scheduler and local IPC.
- service_client.py: desktop snapshots and control requests.
- garden-sprinkler.service: boot/logout systemd user unit.
- controller.py: schedule engine, timed overrides, JSON persistence.
- waveshare_relay.py: RTU-over-TCP adapter and read-only status/discovery command.
- tapo_plugs.py: verified P110M transport, private setup/mapping and opt-in hot-water reconciliation.
- tapo_details.py: read-only telemetry and persistent quarter-hour measurements.
- activity_history.py: bounded event history and 24-hour requested/confirmed timelines.
- phone_server.py, phone_ui.html, phone_ui.css and phone_ui.js: authenticated phone interface and compact group-stop controls.
- system_backup.py: encrypted application backup and fail-closed restore.
- timetable_profiles.py and schedule_overview.py: per-bank profiles and schedule views.
- test_controller.py and test_waveshare_relay.py, and test_app.py, and test_service.py: automated tests. GUI checks require a display and use mocked relay writes.
- test_group_isolation.py, test_tapo_hot_water.py and the other Tapo tests cover independent faults, explicit retry, hot-water reconciliation, telemetry and graphs.

14. Tests

Install the selected environment's requirements first. The controller, transport, service, Tapo, phone and backup suites below do not need a graphical display:

```
.venv/bin/python -m unittest -v \
  test_controller test_waveshare_relay test_service \
  test_group_isolation test_tapo_hot_water test_tapo_plugs \
  test_tapo_details test_tapo_details_service test_tapo_samples \
  test_tapo_timeline test_phone_server test_system_backup
```

Tests cover validation, configuration round-trips and failure preservation, schedule boundaries, weekdays, exact manual/test deadlines, override suppression, clock rollback, independent zones, and shutdown behavior. Protocol tests cover vendor CRC examples, fragmented responses, malformed frames, device exceptions, timeout errors, write/readback validation, and read-only discovery. A loopback TCP server tests controller startup, manual operation, and shutdown over real local sockets. Service tests use private local Unix sockets and fake hardware, including logout-safe client closure, exclusive ownership, restart state, stale-edit rejection, Rain Pause and fault latching. Run `.venv/bin/python -m unittest -v` to include GUI checks with a display. Tests do not contact the physical relay board. Use `.venv-tapo/bin/python` instead when validating the experimental controller environment. Group-isolation tests simulate unreachable appliances, failed ON/OFF and reads, garden/pump faults, independent stop buttons, failed/successful explicit retries, suppressed missed windows and shared-failure shutdown. UI checks cover desktop retry and group controls; phone layout has also been checked in-browser at 320, 360, 412 and 768 CSS pixels, with all three stop buttons on one row, no horizontal overflow and tap heights of at least 44 CSS pixels.